Heart Disease Prediction System

Comprehensive Final Technical Report

Table of Contents

---

1. Abstract

This project presents the design and implementation of a complete machine learning-based healthcare prediction system for detecting heart disease using patient clinical measurements. The system combines

data preprocessing, exploratory data analysis, supervised machine learning algorithms, hyperparameter optimization, explainability techniques, and a full-stack Flask web application.

Five machine learning algorithms were trained and evaluated using Stratified 10-Fold Cross Validation:

- Decision Tree
- Random Forest
- Gradient Boosting
- Support Vector Machine (SVM)
- Logistic Regression

The Random Forest classifier achieved the best overall performance with:

- 94.70% Accuracy
- 96.33% Recall
- 97.30% ROC-AUC

The final model was integrated into a Flask-based web application capable of:

- Receiving patient information
- Predicting heart disease risk
- Generating prediction probabilities
- Displaying personalized recommendations
- Maintaining prediction history

This project demonstrates the integration of machine learning, explainable AI, and full-stack web development for healthcare-oriented applications.

---

2. Executive Summary

Heart disease remains one of the leading causes of mortality worldwide. Early diagnosis is essential for improving treatment outcomes and reducing health risks.

The purpose of this project was to develop an AI-assisted diagnostic support system capable of predicting heart disease using patient medical data.

The system was designed with several key goals:

- High predictive performance
- Medical sensitivity prioritization
- Prevention of data leakage
- Explainability and transparency
- Real-world deployment capability
- User-friendly interface

A complete machine learning workflow was implemented, including:

- Data preprocessing pipelines
- Cross validation
- Model comparison
- Hyperparameter tuning
- Feature importance extraction
- Flask deployment

The Random Forest classifier was selected as the final production model due to its superior performance and strong generalization capability.

---

3. Introduction

Artificial Intelligence and Machine Learning are increasingly being used in healthcare systems to assist in diagnosis, prediction, and medical decision-making.

Machine learning algorithms can identify complex patterns in patient data that may not be immediately obvious using traditional analysis methods.

Heart disease prediction is one of the most important applications of medical AI because:

- Cardiovascular diseases are widespread
- Early detection significantly improves survival rates
- Preventive treatment reduces complications
- AI systems can assist healthcare professionals

This project focuses on building a robust predictive system using structured medical data.

The final system combines:

- Data Science
- Machine Learning
- Explainable AI
- Backend Development
- Frontend Development

into one deployable healthcare prediction platform.

---

4. Problem Statement

Many heart disease patients are diagnosed too late due to:

- Delayed clinical evaluation
- Limited access to specialists
- Ignored early symptoms
- High diagnostic costs

Traditional diagnosis often requires:

- ECG tests
- Stress testing
- Blood analysis
- Medical imaging
- Specialist consultation

The objective of this project is to build a machine learning system capable of predicting heart disease risk using accessible clinical measurements.

The project does NOT replace medical professionals.

Instead, it serves as:

- An AI-assisted screening tool
- A preliminary risk assessment system
- A healthcare support application

---

5. Project Objectives

The major objectives of this project include:

Machine Learning Objectives

- Build a complete ML pipeline
- Train multiple classification algorithms
- Optimize predictive performance
- Prevent data leakage
- Improve medical recall performance
- Generate explainable outputs

Web Application Objectives

- Create a responsive frontend interface
- Build a Flask prediction API
- Integrate ML model with frontend
- Display personalized recommendations
- Store prediction history

Research Objectives

- Compare multiple ML algorithms
- Analyze important medical features
- Study model explainability
- Investigate healthcare AI deployment

---

6. Dataset Description

Dataset Used

The project uses:

`heart_statlog_cleveland_hungary_final.csv`

This dataset combines several famous cardiovascular datasets:

- Cleveland Heart Disease Dataset
- Hungarian Heart Disease Dataset
- Statlog Heart Dataset

Dataset Statistics

Property	Value
Total Records	1190
Features	11
Positive Cases	629
Negative Cases	561
Missing Values	Minimal

Property	Value
Problem Type	Binary Classification

The dataset is relatively balanced, reducing the risk of severe class imbalance problems.

Feature Description

Feature	Type	Description
age	Numeric	Patient age
sex	Binary	Male/Female
chest pain type	Categorical	Type of chest pain
resting bp s	Numeric	Resting blood pressure
cholesterol	Numeric	Serum cholesterol
fasting blood sugar	Binary	Fasting blood sugar status
resting ecg	Categorical	ECG result
max heart rate	Numeric	Maximum heart rate
exercise angina	Binary	Exercise-induced angina
oldpeak	Numeric	ST depression
ST slope	Categorical	Slope of ST segment
target	Binary	Heart disease presence

---

7. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed before model training to better understand the dataset.

The following analyses were conducted:

Dataset Shape Analysis

The dataset dimensions were examined using:

```
print(df.shape)
```

Missing Value Analysis

Missing values were identified using:

```
print(df.isnull().sum())
```

Duplicate Record Analysis

Duplicate rows were checked and analyzed.

Target Distribution Analysis

A target distribution chart was generated to analyze class balance.

Correlation Analysis

A heatmap was generated using:

```
df.corr()
```

This helped identify:

- Strongly correlated variables
- Potential feature relationships
- Medical trends

Distribution Analysis

Histograms and boxplots were generated for:

- Age
- Cholesterol
- Blood Pressure
- Oldpeak
- Maximum Heart Rate

These visualizations helped detect:

- Outliers
- Skewness
- Data spread

---

8. Data Preprocessing Pipeline

A complete preprocessing system was implemented using:

- Pipeline
- ColumnTransformer
- SimpleImputer
- StandardScaler
- OneHotEncoder

This approach ensures:

- Clean workflow
- No data leakage
- Reproducibility
- Consistent preprocessing

Numeric Feature Processing

Numeric features included:

- Age
- Resting BP
- Cholesterol
- Max Heart Rate
- Oldpeak

Processing Steps

1. Median Imputation
2. Standard Scaling

Median imputation was selected because it is more robust to outliers than mean imputation.

Categorical Feature Processing

Categorical features included:

- Chest Pain Type
- Resting ECG
- ST Slope

Processing Steps

1. Most Frequent Imputation
2. One-Hot Encoding

OneHotEncoding converts categorical values into binary vectors suitable for machine learning models.

Binary Feature Processing

Binary features included:

- Sex
- Fasting Blood Sugar
- Exercise Angina

These features were passed directly into the model pipeline.

---

9. Feature Engineering Strategy

Feature engineering focused on:

- Proper feature categorization
- Encoding categorical variables
- Scaling continuous variables
- Maintaining consistent feature ordering

The project also extracted processed feature names after OneHotEncoding for:

- Feature importance analysis
- Explainability
- Frontend integration

---

10. Train/Test Splitting

The dataset was divided into:

- 80% Training Data
- 20% Testing Data

using:

```
train_test_split(..., stratify=y)
```

Stratification ensures that:

- Disease ratios remain balanced
- Evaluation remains fair
- Class distribution consistency is maintained

A fixed random state (`random_state=42`) was used for reproducibility.

---

11. Cross Validation Methodology

The project used:

Stratified 10-Fold Cross Validation

Configuration:

```
StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=42
)
```

Cross validation improves:

- Reliability
- Generalization
- Stability of evaluation

Each model was trained and tested 10 times on different folds.

The final score represents the average across all folds.

12. Machine Learning Algorithms

Five machine learning algorithms were implemented and compared.

---

1. Decision Tree Classifier

Characteristics

- Tree-based algorithm
- Highly interpretable
- Explainable rules
- Easy visualization

Usage in Project

- Baseline explainable model
- Rule extraction
- Tree visualization

---

2. Random Forest Classifier

Characteristics

- Ensemble learning algorithm
- Collection of multiple decision trees
- Reduces overfitting
- Strong predictive capability

Configuration

```
n_estimators=300
```

Usage in Project

- Final production model
- Best performing classifier

---

3. Gradient Boosting Classifier

Characteristics

- Sequential tree boosting
- Corrects previous errors
- High predictive performance

---

4. Support Vector Machine (SVM)

Configuration

- RBF Kernel
- probability=True

Characteristics

- Effective for complex boundaries
- Strong classification capability
- Computationally expensive

---

5. Logistic Regression

Characteristics

- Linear classification model
- Fast training
- Interpretable coefficients
- Baseline statistical model

---

13. Hyperparameter Optimization

Hyperparameter optimization was performed using:

GridSearchCV

Parameters Tuned

Parameter	Values
max_depth	3, 5, 7, None
min_samples_split	2, 10, 20
criterion	gini, entropy
ccp_alpha	0.0, 0.01, 0.05

Optimization Metric

The optimization process used:

```
scoring='recall'
```

This decision was made because:

- False negatives are dangerous in healthcare
- Missing a diseased patient is high-risk
- Recall measures disease detection capability

---

14. Evaluation Metrics

The following evaluation metrics were used:

Metric	Purpose
Accuracy	Overall correctness
Precision	Positive prediction quality
Recall	Disease detection ability
F1 Score	Precision/Recall balance
ROC-AUC	Overall classifier quality

Recall Importance in Healthcare

In medical systems:

False Negatives are significantly more dangerous than False Positives.

A false negative means:

- The patient actually has heart disease
- But the system predicts healthy

This may delay treatment and increase mortality risk.

Therefore:

Recall was prioritized throughout model optimization.

---

15. Model Performance Analysis

Cross Validation Results

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Random Forest	94.70%	93.83%	96.33%	95.05%	97.30%
SVM (RBF)	87.98%	87.47%	90.30%	88.81%	93.46%
Gradient Boosting	88.99%	89.64%	89.66%	89.60%	94.94%
Decision Tree	89.49%	91.29%	88.71%	89.91%	89.54%
Logistic Regression	84.53%	84.41%	86.95%	85.60%	91.71%

Best Model Selection

The Random Forest classifier achieved:

- Highest Recall
- Highest ROC-AUC
- Highest overall performance
- Excellent generalization
- Strong robustness

Therefore:

The Random Forest model was selected as the final deployed production model.

---

16. Feature Importance Analysis

Feature importance extraction was performed using the trained Random Forest classifier.

Top Important Features

Rank	Feature	Importance
1	ST slope_1	12.75%
2	Oldpeak	10.72%
3	Max Heart Rate	10.57%
4	Cholesterol	10.43%
5	Chest Pain Type 4	9.93%
6	Age	8.94%
7	Resting BP (s)	7.98%
8	ST slope_2	6.89%
9	Exercise Angina	6.46%
10	Sex	4.13%

Medical Interpretation

The extracted feature importances align with known medical knowledge:

- ST slope abnormalities are associated with cardiovascular problems
- High cholesterol is a major risk factor
- Exercise angina strongly correlates with heart disease
- Age significantly affects cardiovascular risk

This increases confidence in the model's clinical relevance.

---

17. Explainability and Interpretability

Explainability was incorporated into the system through:

Decision Tree Visualization

The Decision Tree model was visualized graphically using:

```
plot_tree()
```

Rule Extraction

Human-readable decision rules were generated using:

```
export_text()
```

This improves:

- Transparency
- Interpretability
- Educational value

---

18. Visualizations Generated

The project generated multiple visualization outputs.

Visualization	Purpose
Target Distribution	Dataset balance
Correlation Heatmap	Feature relationships
Histograms	Distribution analysis
Boxplots	Outlier detection
ROC Curves	Model comparison
Confusion Matrices	Classification analysis
Feature Importance Chart	Explainability
Decision Tree Visualization	Model interpretation
Model Comparison Chart	Performance comparison

All graphs were exported as high-resolution PNG files.

19. Flask Web Application Architecture

The machine learning model was integrated into a Flask web application.

Frontend Technologies

- HTML
- CSS
- JavaScript

Backend Technologies

- Flask
- Joblib
- Pandas

Main Flask Routes

Route	Purpose
/	Homepage
/predict	Prediction API
/recommendations	Recommendations page

20. Frontend System Design

The frontend interface was designed with:

- Responsive layout
- Interactive medical form
- Input validation
- Prediction history
- Dynamic recommendations
- Modern UI/UX design

The frontend collects:

- Age
- Sex
- Chest Pain Type
- Blood Pressure
- Cholesterol
- ECG Results
- Additional medical measurements

---

21. Backend Prediction Flow

The backend prediction flow operates as follows:

1. User submits medical form
2. Flask receives POST request
3. Data converted to DataFrame
4. Random Forest pipeline preprocesses input
5. Prediction generated
6. Probability calculated
7. JSON response returned
8. Frontend displays recommendations

---

22. Risk Classification Logic

Prediction probabilities were classified into:

Probability Range	Risk Level
< 30%	Low Risk
30% - 60%	Mild Risk
> 60%	High Risk

Each risk category displays personalized recommendations.

---

23. Model Serialization and Deployment

The final trained model was serialized using:

```
joblib.dump()
```

Saved Files

File	Description
heart_disease_model.pkl	Full trained model pipeline
feature_names.pkl	Processed feature names
decision_tree_rules.txt	Tree rules

Deployment Benefits

Model serialization enables:

- Fast loading
- Reusability
- Real-time predictions
- Deployment without retraining

---

24. Challenges Encountered

Several technical challenges were encountered during development.

Challenge 1 — Data Leakage

Problem

Improper preprocessing before splitting can leak information from test data.

Solution

Complete preprocessing was placed inside Scikit-learn Pipelines.

Challenge 2 — Frontend Feature Matching

Problem

Frontend form names initially differed from model feature names.

Solution

Frontend field names were aligned exactly with model features.

Challenge 3 — Model Selection

Problem

Some models achieved good accuracy but lower recall.

Solution

Recall was prioritized over raw accuracy.

---

25. Solutions Implemented

The following solutions improved system quality:

- Pipeline-based preprocessing
- Stratified cross validation
- Hyperparameter optimization
- Random Forest ensemble learning
- Explainability integration
- Flask deployment architecture
- Frontend validation system

---

26. System Workflow (End-to-End)

The complete workflow operates as follows:

1. User enters patient data
2. Frontend validates inputs
3. Data sent to Flask backend
4. Backend loads trained Random Forest model
5. Pipeline preprocesses input features

6. Model predicts heart disease probability
7. Risk level generated
8. Personalized recommendations displayed
9. Prediction history stored locally

---

27. Security and Validation

Several validation mechanisms were implemented:

Frontend Validation

- Numeric range validation
- Required field validation
- Data type validation

Backend Validation

- Safe DataFrame conversion
- Error handling
- Type conversion protection

---

28. Future Improvements

Potential future improvements include:

- SHAP Explainability
- Database integration
- User authentication
- PDF report generation
- Cloud deployment
- Mobile application support
- Deep learning integration
- Real-time dashboards
- API security enhancements
- XGBoost implementation

---

29. Conclusion

This project successfully demonstrates the development of a complete AI-powered healthcare prediction system.

The final Random Forest classifier achieved excellent performance and demonstrated strong capability for detecting heart disease risk.

The system integrates:

- Machine Learning
- Data Science
- Explainable AI
- Flask Backend Development
- Frontend Engineering

into one production-ready healthcare application.

The project highlights the potential of AI-assisted healthcare systems for:

- Early disease detection
- Medical risk assessment
- Clinical decision support
- Healthcare technology advancement

---

30. References

1. Scikit-learn Documentation
2. Flask Documentation
3. Pandas Documentation
4. NumPy Documentation
5. Matplotlib Documentation
6. UCI Machine Learning Repository
7. Heart Disease Cleveland Dataset
8. Statistical Learning Resources

Final Disclaimer

This system is intended for educational and research purposes only and should not replace professional medical diagnosis or consultation with qualified healthcare providers.